

SVCE Biotechnology · Talent Portal

An interactive, single-page version of the Department of Biotechnology placement brochure (Sri Venkateswara College of Engineering). It lets HR / recruiters understand the department and **search, filter, and shortlist** the graduating cohort by skill and field of interest, with links to each student's résumé.

This document is written for **whoever maintains the site next year** — it explains what the site does, how it's built, and exactly how to update the content for a new batch.

1. What the site does (features)

Understand the department

- Brochure-style **cover** and a **table of contents** that jumps to each section.
- **About** — department overview, an "At a glance" facts card, the **Head of Department's note** (photo + full message), and **SVCE Global Connect**.
- **Faculty** — all faculty with photos and research areas.
- **Facilities & Equipment** — real lab equipment photos, grouped by theme.
- **Curriculum** — the full **B.Tech** and **M.Tech** syllabus, semester by semester (tabbed accordion).
- **Recruiters & PG Admissions** — logos of past recruiters and PG destinations.
- **Recruit With Us** — the placement process, guidelines, the details recruiters should share, and a **Contact us** block with the placement-cell emails.

Find the right candidate (the Candidate Directory)

- **Skill search** — free-text search across name, skills, and fields of interest (e.g. `PCR`, `Python`, `molecular docking`). Matching skills are highlighted on each card.
- **Filters** — *Program* (B.Tech / M.Tech), *Field of interest*, and *Skill area*. Selecting a filter snaps the view back to the top of the directory.
- **Best-match ordering** — when searching, candidates are ranked by how many search terms they match.
- **M.Tech candidates** are grouped at the end of the list and carry a visible **M.Tech** badge.
- Each card shows photo, name, email, interest tags, skill chips, and a **View résumé** link (opens the student's Google Drive folder).

Everywhere

- Fully **responsive** (desktop / tablet / mobile).
 - **Light & dark themes** — follows the visitor's system theme, with a manual toggle. A `?theme=light` or `?theme=dark` URL parameter forces a theme (useful for sharing a specific look).
-

2. Tech & architecture

Deliberately simple so it's easy to hand over and cheap to host.

- **Static site. No build step, no framework, no backend.** Just HTML, CSS, and vanilla JavaScript. You can open `index.html` directly or drop the folder on any static host.
- **Single page.** Everything lives in `index.html` as stacked `<section>`s; the top nav uses in-page anchors with scroll-spy highlighting.
- **Shared chrome is injected once.** The header, nav, footer, theme toggle, and scroll-spy are built by `js/layout.js` and injected into `<div id="app-header">` / `<div id="app-footer">`, so there's a single place to edit them.
- **Data-driven.** All the content that changes year to year lives in plain data files (`data/*.js`) and a couple of arrays inside `index.html`. The UI reads those and renders itself — you rarely touch the layout code.
- **Client-side search & filtering.** `js/candidates.js` reads the candidate list and does all search/filter/sort in the browser. No server or database.
- **Theming via CSS variables.** Colors are CSS custom properties defined for light and dark; components reference the variables, so the whole palette is centralized at the top of `css/styles.css`.

Why static? The dataset is small and fixed for the year, everything is public, and a static site needs no maintenance, has no security surface, and hosts free on Vercel/Netlify.

3. Project structure

```
svce-biotech-portal/
├─ index.html           ← the whole page (all sections) + faculty/facilities/curricu
├─ css/
│  └─ styles.css       ← all styles + the color palette (CSS variables) at the top
├─ js/
│  ├─ layout.js        ← header, nav, footer, theme toggle, scroll-spy (shared chro
│  └─ candidates.js    ← candidate directory: search, filters, sorting, card render
├─ data/
│  ├─ candidates.js    ← THE candidate list (window.CANDIDATES) + filter options (w
│  └─ partners.js      ← recruiters & PG-admission universities (name + logo)
├─ img/
│  ├─ brand/           ← SVCE wordmark, seal, department building photo
│  ├─ faculty/         ← faculty portraits + hod.jpg
│  ├─ facilities/      ← equipment / animal-house photos
│  ├─ candidates/      ← one photo per candidate (filename = slug of the name)
│  └─ logos/
│     ├─ recruiters/   ← recruiter logos (transparent PNG)
│     └─ universities/ ← PG-destination logos (transparent PNG)
├─ vercel.json          ← Vercel config (clean URLs)
├─ netlify.toml         ← Netlify fallback config
├─ .vercelignore / .gitignore
└─ README.md
```

4. Updating the content for next year

Most updates are just editing data files and dropping in images. No coding required beyond following the shapes below.

4.1 Candidates — `data/candidates.js`

This is the single source of truth for the directory. It defines two globals:

- `window.FACETS` — the options shown in the filter panel (Program, Field of interest, Skill area). Keep these stable unless you're introducing a genuinely new category.
- `window.CANDIDATES` — an array with **one object per student**. Shape:

```
{
  "name": "Akshaya N",
  "program": "B.Tech (UG)",           // or "M.Tech (PG)" → M.Tech shows a badge &
  "email": "student@example.com",
  "photo": "img/candidates/akshaya-n.jpg",
  "fieldsDisplay": ["Computational Biology", "Bioprocess Engineering"], // SHOWN on t
  "fieldsOfInterest":["Bioinformatics / Computational Biology", "Bioprocess Engineering
  "skillAreas":      ["Cell Culture", "Programming & Data"], // drives the SKILL-AREA
  "labType":         ["Wet lab", "Computational"],           // legacy field; the lab-
  "skills":          ["Cell staining", "Python", "R"],       // SHOWN as chips + search
  "resumeUrl": "https://drive.google.com/drive/folders/..." // opens in a new tab; le
}
```

Key idea — display vs. filter are decoupled: - `fieldsDisplay` and `skills` are what the visitor sees — keep them in the student's own words. - `fieldsOfInterest` and `skillAreas` are the *canonical* buckets that power the filters — they must match the values in `FACETS`. This lets the tags read naturally while the filters still work.

To add / update a student: 1. Add or edit their object in `window.CANDIDATES`. 2. Add their photo to `img/candidates/` (see image guidelines below) and point `photo` at it. 3. Put the résumé folder link in `resumeUrl`. 4. For a postgraduate, set `program` to `"M.Tech (PG)"` — the site badges them and lists them after the B.Tech students automatically.

4.2 Recruiters & PG universities — `data/partners.js`

Two arrays, `window.RECRUITERS` and `window.UNIVERSITIES`, each `{ name, logo }`:

```
{ name: "Biocon", logo: "img/logos/recruiters/biocon.png" }
```

Add the logo (transparent PNG works best) under `img/logos/recruiters/` or `.../universities/` and add the entry. They render automatically on the home page strip and the Recruit section.

4.3 Faculty — array inside `index.html`

Search `index.html` for `var FACULTY`. Each entry is `[name, title, research areas, photo-slug]`:

```
['Dr. E. Nakkeeran', 'Professor & Head', 'Bioprocess Engineering, Downstream Processing,
```

Add the portrait as `img/faculty/<slug>.jpg` and add the row.

4.4 Facilities — array inside `index.html`

Search for `var GROUPS`. Each group maps a heading to a list of `[label, photo-slug]` pairs; photos live in `img/facilities/<slug>.jpg`.

4.5 Curriculum — arrays inside `index.html`

Search for `var BTECH` and `var MTECH`. Each semester is `['Semester I', [theory...], [practical...]]`. Edit the lists to match the current syllabus.

4.6 About text, HoD note, stats, contact emails

All in `index.html`: - **About / Global Connect / HoD note** — the `#about` section (plain prose you can edit directly). - **"At a glance" facts** — the `.facts` list in `#about`. - **Contact emails** — the `#contact` block in the Recruit section, and the footer's "Placement Cell" list is in `js/layout.js`.

5. Image guidelines

- **Candidate & faculty photos:** square, **face-centred head-and-shoulders**, ~440×440 px JPG. The site displays them in circles (`object-fit: cover`), so anything that isn't a centred square will look cropped or off-centre. If you have tall/full-body source photos, crop to the face first.
 - **Logos:** transparent-background **PNG**; they're shown on white tiles.
 - **Filenames = the slug** of the name: lowercase, spaces → hyphens (e.g. `Akshaya N` → `akshaya-n.jpg`).
-

6. How search & filters work (for reference)

- **Search** matches the typed terms against each candidate's **name, skills, and fields of interest**; results are ranked by number of matches, and matched skill chips are highlighted.
- **Field-of-interest filter** checks a candidate's `fieldsOfInterest` (canonical) against the ticked options.
- **Skill-area filter** checks `skillAreas`.
- **Program filter** checks `program`.
- Selecting any filter scrolls back to the top of the directory so the list doesn't jump around.

If you add a brand-new field-of-interest or skill-area value, add it to `FACETS` **and** use the exact same string in the relevant candidates' `fieldsOfInterest` / `skillAreas`.

7. Run locally

Open `index.html` in a browser, or serve the folder (recommended, so relative paths behave):

```
npx serve .
```

Then visit the printed `http://localhost:####` address.

8. Deploy (Vercel — private source, public site)

The project is already linked to a Vercel project. From the folder:

```
npx vercel login          # one-time authentication
npx vercel deploy --prod  # pushes to production, prints the live URL
```

- The CLI uploads only to your Vercel account — **no GitHub repo**, so the source stays private.
- The **site** is public at its `...vercel.app` URL (that's the point — recruiters open it). To also gate the site behind a login, turn on **Deployment Protection** in the Vercel project settings.
- `netlify.toml` is included if you ever prefer Netlify (drag-and-drop the folder, or `netlify deploy`).

9. Notes & gotchas

- **Résumé links** point to each student's **Google Drive folder**. Make sure those folders are shared as "anyone with the link can view", otherwise recruiters hit a sign-in wall.
- **Photos must be shared/owned by you** and stored in the repo (`img/candidates/`) — the site does not pull images from Drive at runtime.
- **Everything is client-side and public**. Don't put anything in the data files you wouldn't want on the open web (the site currently shows student names, emails, photos, interests, and résumé links — confirm the students consented to this being public, or enable Deployment Protection).
- **No build to break**. If a card looks wrong, it's almost always a typo in `data/candidates.js` (e.g. a filter value that doesn't exactly match a `FACETS` entry) — check the browser console.